

Module 15) Advance Python Programming

8. Method Overloading and Overriding

- **Method overloading: defining multiple methods with the same name but different parameters.**

What is Method Overloading?

- **Method overloading** means defining **multiple methods with the same name but different parameters** (different number or types of parameters) within the same class.
- In some programming languages (like Java or C++), this is supported natively.

How about Python?

- **Python does not support traditional method overloading** directly.
- If you define multiple methods with the same name, the **last one defined overwrites the previous ones**.

How to Achieve Similar Behavior in Python?

1. Use Default Arguments

- Define a single method with default values for parameters.

```
class Example:
```

```
    def greet(self, name=None):
```

```
        if name:
```

```
        print(f"Hello, {name}!")
    else:
        print("Hello!")
```

- **Method overriding: redefining a parent class method in the child class.**

What is Method Overriding?

- **Method overriding** occurs when a **child class redefines a method** that is already defined in its **parent class**.
- It allows the child class to provide a **different implementation** of the method.
- This is a key feature of **polymorphism** in object-oriented programming.

Why Use Method Overriding?

- To **modify or extend** the behavior of the parent class method.
- To customize functionality specific to the child class.

How Does Method Overriding Work?

- The child class defines a method with the **same name and parameters** as a method in the parent class.
- When the method is called on a child class object, the **child's version is executed**, not the parent's.

Example:

```
class Parent:
```

```
def greet(self):  
    print("Hello from Parent")
```

```
class Child(Parent):  
    def greet(self):  
        print("Hello from Child")
```

```
obj = Child()
```

```
obj.greet() # Output: Hello from Child
```